

# ECS 150 - System Calls

---

*Prof. Joël Porquet-Lupine*

UC Davis - SQ25



# C Standard Library

## C program example

```
#include <fcntl.h>
#include <stdio.h>
...
int main(int argc, char *argv[])
{
    int fd;
    char buf[4];

    if (argc < 2)
        exit(EXIT_FAILURE);

    memset(buf, 0, sizeof(buf));

    fd = open(argv[1], O_RDONLY);
    if (fd < 0) {
        perror("open");
        exit(EXIT_FAILURE);
    }
    read(fd, buf, sizeof(buf));
    close(fd);

    printf("Executable detection... ");
    if (buf[0] == 0x7F && buf[1] == 0x45
        && buf[2] == 0x4C && buf[3] == 0x46)
        printf("ELF");
    else if (buf[0] == '#' && buf[1] == '!')
        printf("script");
    else
        printf("Not an executable!");
    printf("\n");

    return 0;
}
```

exec\_detector.c

## Execution

```
$ make exec_detector
cc -o exec_detector exec_detector.c
```

```
$ ./exec_detector
$ ./exec_detector /path/to/nofile
open: No such file or directory
```

```
$ ./exec_detector exec_detector
Executable detection... ELF
```

```
$ ./exec_detector exec_detector.c
Executable detection... Not an executable!
```

```
$ ./exec_detector /bin/firefox
Executable detection... script
```

```
$ cat /bin/firefox
#!/bin/sh
exec /usr/lib/firefox/firefox "$@"
```

```
$ ./exec_detector /usr/lib/firefox/firefox
Executable detection... ELF
```

# C Standard Library

## Library functions

```
#include <fcntl.h>
#include <stdio.h>

...
memset(buf, 0, sizeof(buf));
...
fd = open(argv[1], O_RDONLY);
if (fd < 0) {
    perror("open");
    exit(EXIT_FAILURE);
}
read(fd, buf, sizeof(buf));
close(fd);
...
printf("Executable detection... ");
...
printf("\n");
```

exec\_detector.c

## Declaration

- Access via inclusion of headers
  - Function prototypes
  - Global variables (e.g. `errno`)
  - Type definitions (e.g. `size_t`)
  - Macros (e.g., `O_RDONLY`)

## Definition

- Actual code via library
- Linked at compile-time
- Dynamically loaded at runtime

## Categories of `libc` functions

1. No privileged operation to perform
2. Always needs to request privileged operation from OS
  - Known as **system call**, or *syscall*
3. Sometimes needs to request privileged operation from OS

# C Standard Library

## 1. Regular functions

### Usage example

```
char buf[4];  
...  
memset(buf, 0, sizeof(buf));
```

exec\_detector.c

- `memset()` only needs to have access to array `buf`
  - `buf` is already part of the application's memory (here, defined in stack)
  - No need for any special operations

### Implementation example

- Generic C implementation

```
void *memset(void *s, int c, size_t count)  
{  
    char *xs = s;  
  
    while (count--)  
        *xs++ = c;  
  
    return s;  
}
```

- See MUSL's [memset\(\)](#) optimized implementation if you're curious!

# C Standard Library

## 2. (Always) privileged functions

### Usage example

```
fd = open(argv[1], O_RDONLY);  
...  
read(fd, buf, sizeof(buf));  
close(fd);
```

exec\_detector.c

- Need for special privileges: e.g.,
  - Verify that file exists on a physical medium accessible by computer (e.g., hard-drive, SD card, network, etc.)
  - Check that current user has permission to open it with specified mode
  - Actually read file's data from physical medium
- Sensitive operations passed to OS
  - Accessible via *syscalls*

### Implementation example

- For example in GNU/Linux

```
#include <unistd.h>  
#include <sys/syscall.h> // Syscall code numbers  
  
ssize_t read(int fd, void *buf,  
             size_t count)  
{  
    long r = syscall3(SYS_read,  
                      fd, buf, count);  
  
    if (r < 0) {  
        errno = -r;  
        return -1;  
    }  
    return r;  
}
```

- Syscall for x86\_64 processors

```
static __inline long __syscall3(long n, long a1,  
                                long a2, long a3)  
{  
    unsigned long ret;  
    __asm__ __volatile__ ("syscall" : "=a"(ret)  
                           : "a"(n), "D"(a1),  
                           "S"(a2), "d"(a3)  
                           : "rcx", "r11",  
                           "memory");  
    return ret;  
}
```

# C Standard Library

## 3. (Sometimes) privileged functions

### Usage example

```
printf("Executable detection... ");  
...  
printf("\n");
```

exec\_detector.c

- `printf()` prints to `stdout` which is (generally) internally buffered
  - Flushed when buffer is full, or when encountering `\n`
- Flushing requires to OS to actually write the characters
  - Use of (syscall) function `write()`

### Printf vs write

```
printf("Hello ");  
sleep(2);  
printf("world!\n");  
  
write(STDOUT_FILENO, "Hello ", 6);  
sleep(2);  
write(STDOUT_FILENO, "world!\n ", 7);
```

printf\_write.c

```
$ ./printf_write  
<wait 2 sec>  
Hello world!  
Hello <wait 2 sec>world!
```

# System calls

## Definition

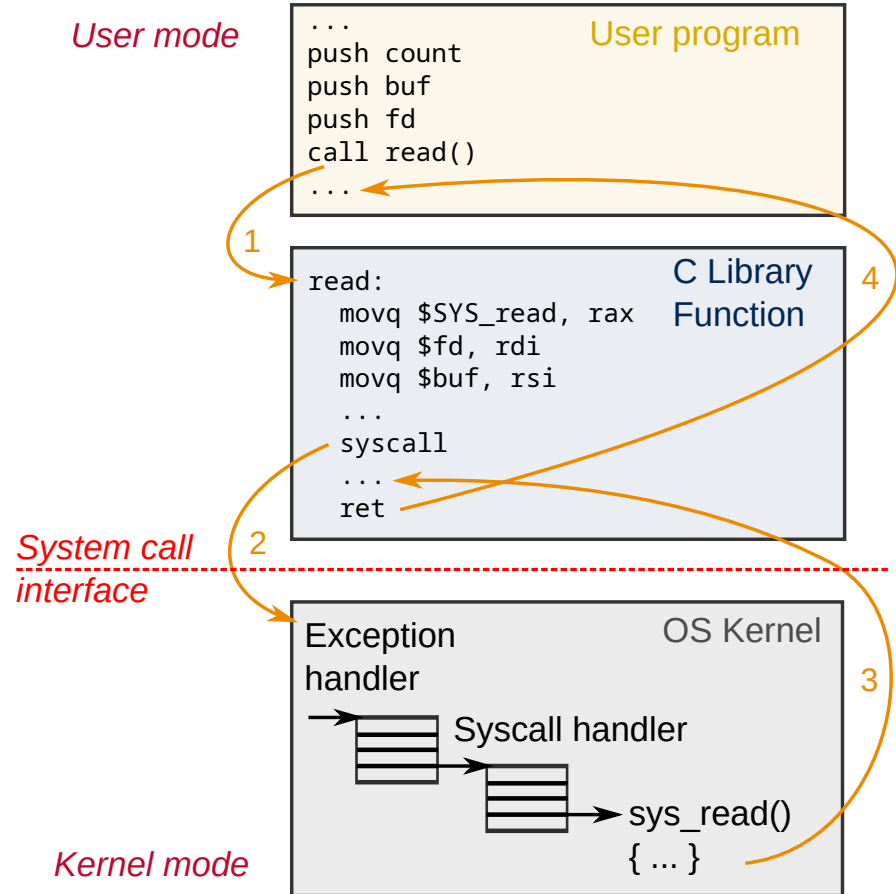
- Specific CPU instruction
- Immediate transfer of control to *kernel* code

## Purpose

- Secure API between user applications and OS kernel

## Main categories

- Process management
- Files and directories
- Pipes
- Signals
- Memory management

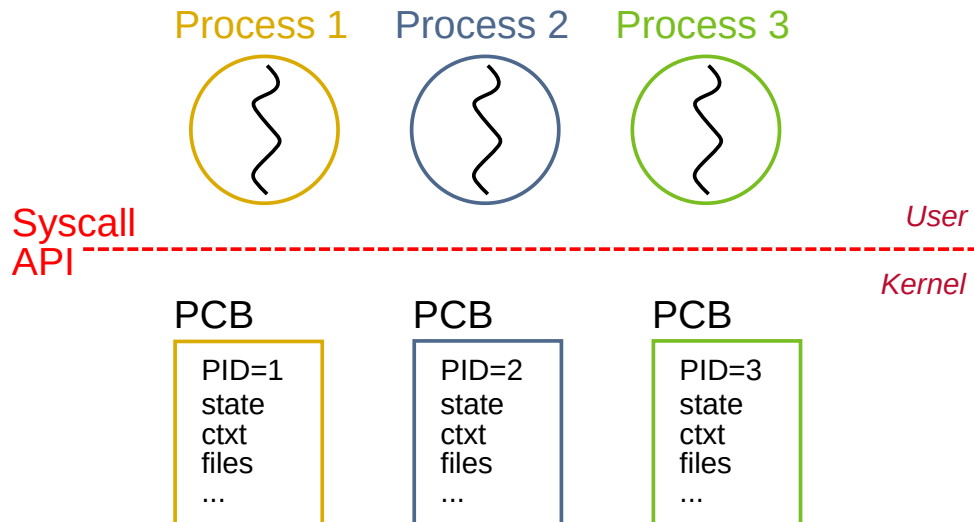


# Process management

## Definition of a process

### A process is a program in execution

- Each process is identified by its *Process ID* (PID)
- Each process runs its own memory space
- Each process is represented in the OS by a *Process Control Block* (PCB)
  - Data structure storing information about process
  - PID, state, CPU register copies for context switching, open files, etc.





# Process management

---

## Main related functions/syscalls

- Process creation and execution
  - `fork()`: Create a new (clone) process
  - `exec()`: Change executed program within running process
- Process termination
  - `exit()`: End running process
  - `wait()/waitpid()`: Wait for a child process and collect exit code
- Process identification
  - `getpid()`: Get process PID
  - `getppid()`: Get parent process PID

# Process management

## fork()

- Running process gets cloned into a *child* process
- Child gets an (almost) identical copy of *parent*
  - Same open files, command line arguments, memory, stack, etc.
- Child "resumes" at the `fork()` as well

## Example

```
int a = 42;

int main(int argc, char *argv[])
{
    int b = 23;

    printf("Hello world!\n");
    fork();
    printf("My favorite number is %d.\n",
        argc + a + b);

    return 0;
}
```

fork\_101.c

```
$ ./fork_101
Hello world!
My favorite number is 66.
My favorite number is 66.
```

# Process management

## Distinguishing between parent and child

- `fork()` returns a value
  - *PID of the child* to the parent
  - *zero* to the child
  - *-1* to the parent in case of error

## Example

```
int main(void)
{
    pid_t pid;

    pid = fork();
    if (pid > 0)
        printf("I'm the parent!\n");
    else if (pid == 0)
        printf("I'm the child!\n");
    else
        printf("I'm the initial process! "
               "But something went wrong...\n");

    printf("I'm here now, bye!\n");
    return 0;
}
```

fork\_201.c

```
$ ./fork_201
I'm the parent!
I'm here now, bye!
I'm the child!
I'm here now, bye!
```

- Output is not guaranteed

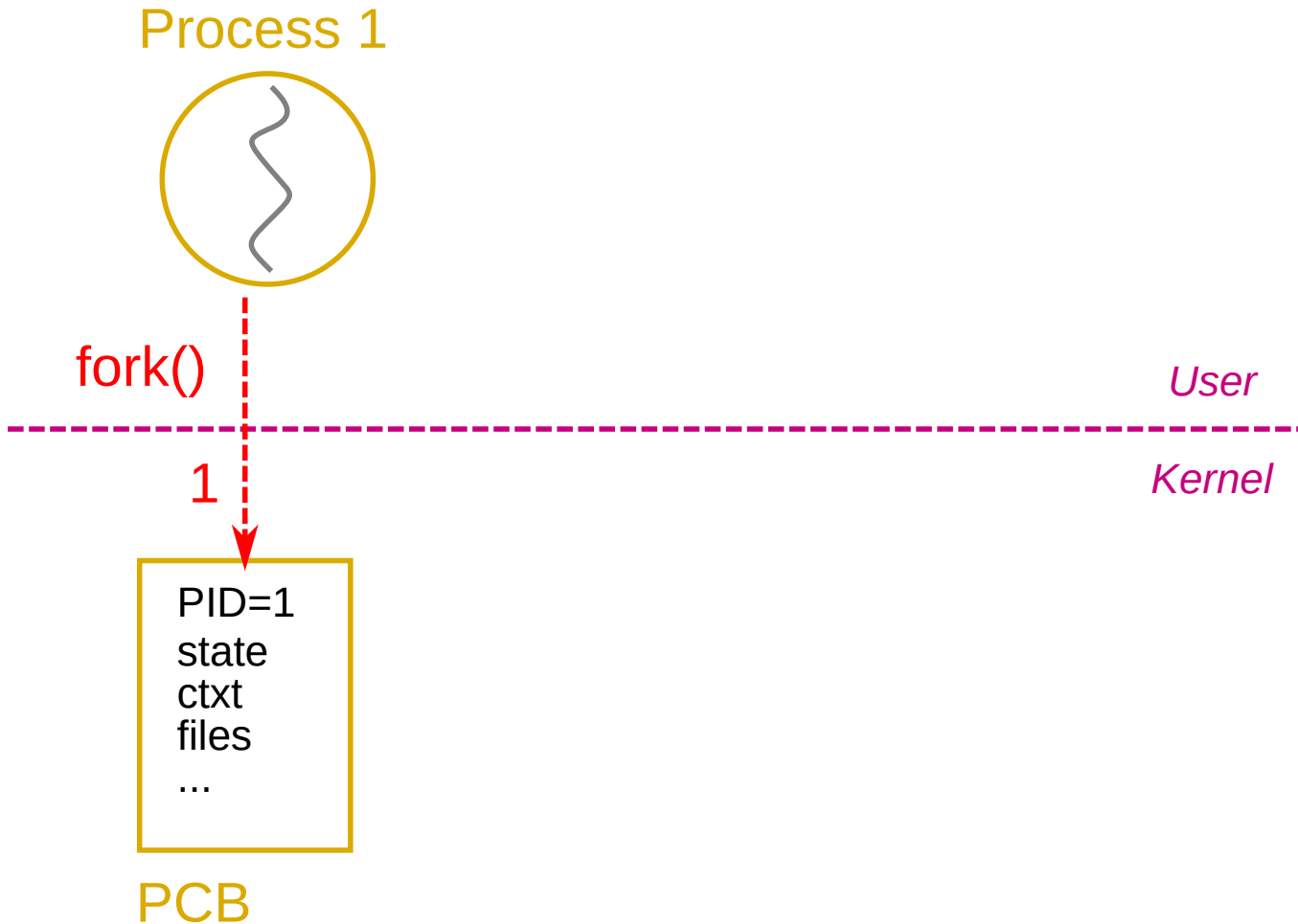
```
I'm the child!
I'm here now, bye!
I'm the parent!
I'm here now, bye!
```

```
I'm the parent!
I'm the child!
I'm here now, bye!
I'm here now, bye!
```

- Parent and child are independent processes
- Scheduling is up to OS

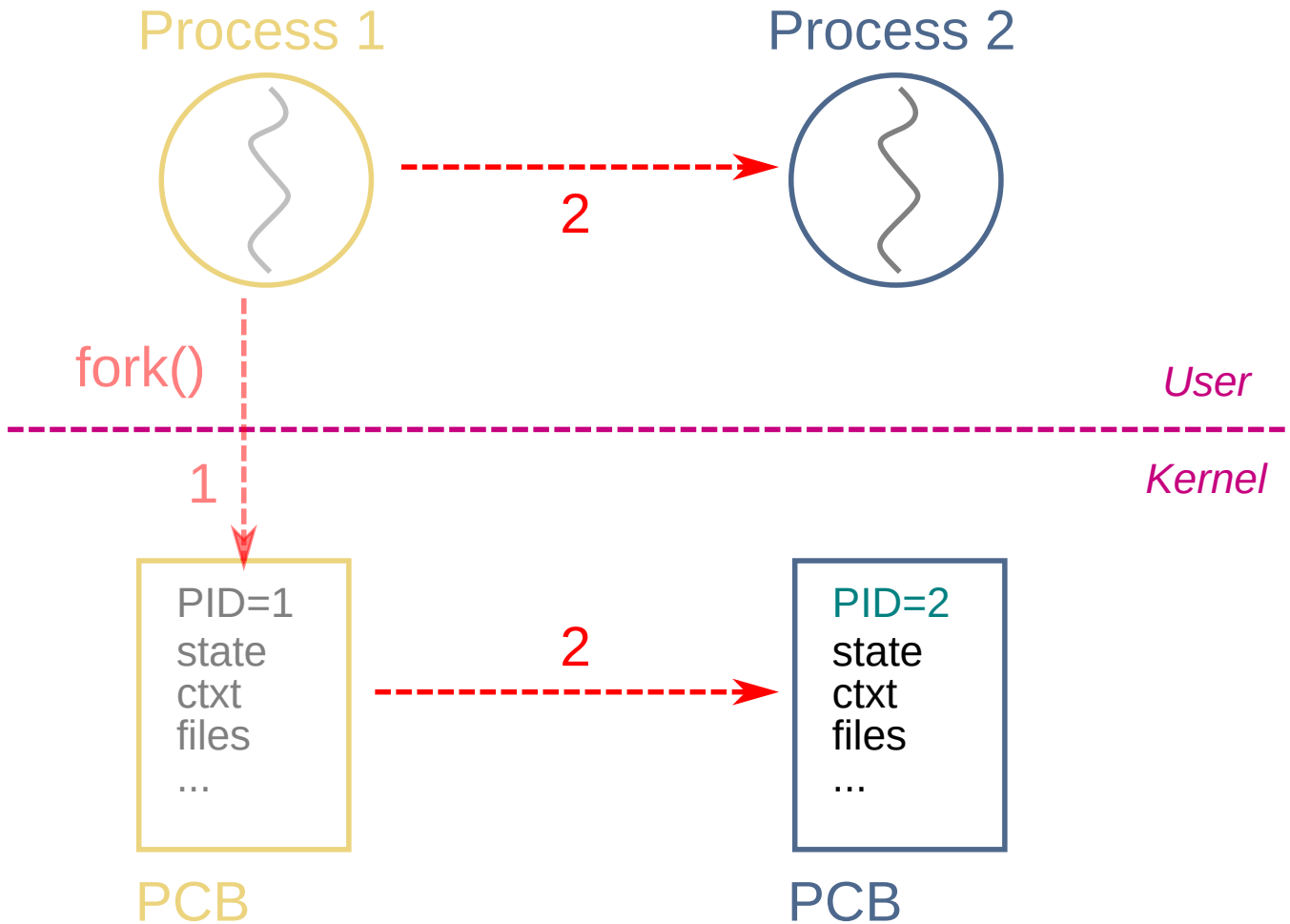
# Process management

Fork illustrated (1/3)



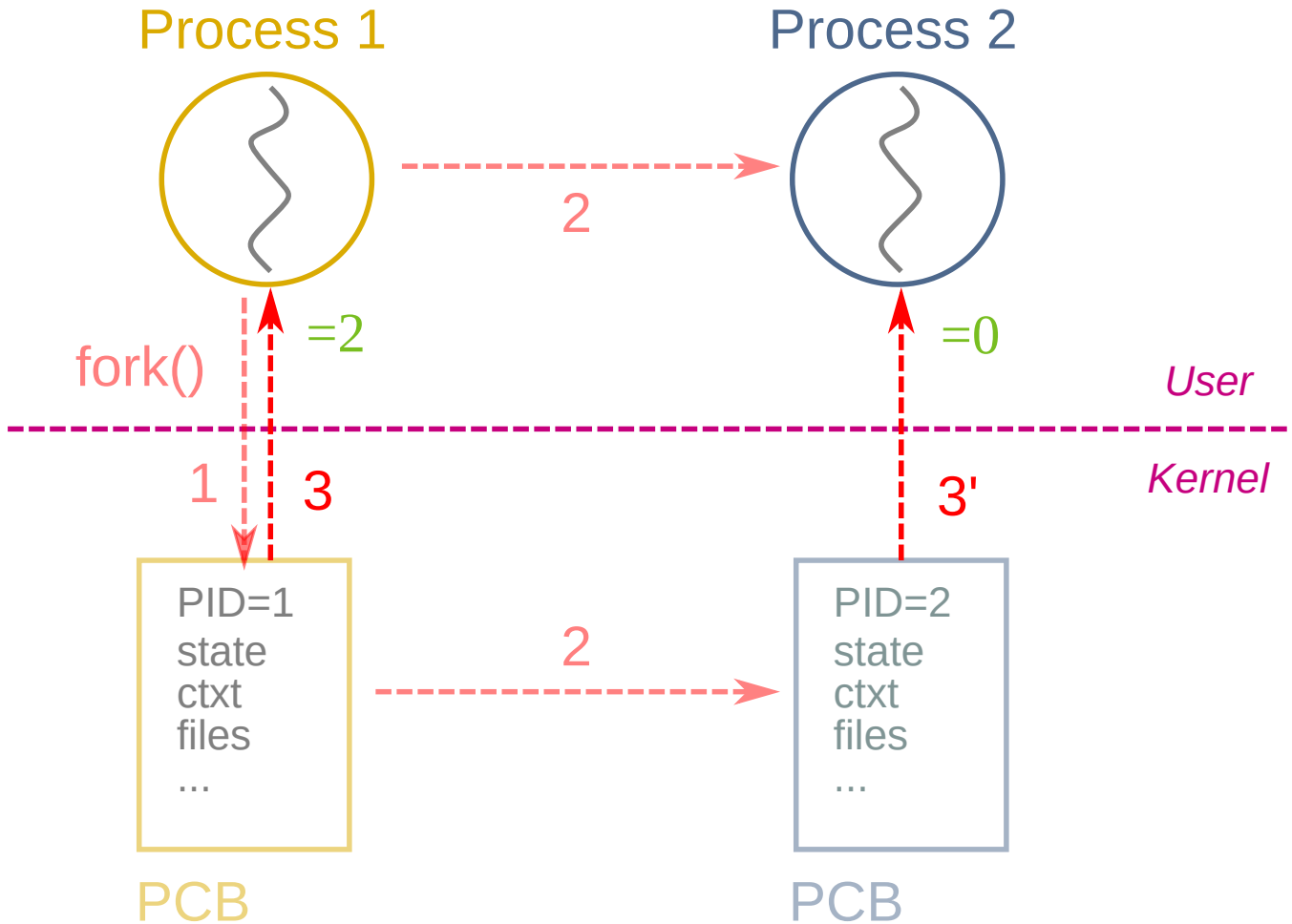
# Process management

Fork illustrated (2/3)



# Process management

Fork illustrated (3/3)



# Process management

## exec()

- Current *process* starts executing another *program*
- Family of functions, with slight variations
  - `exec[lv]p?e?()` (see man page for details)

## Example

```
int main(void)
{
    char *cmd = "/bin/echo";
    char *args[] = { cmd, "ECS150", NULL};
    int ret;

    printf("Hi!\n");

    ret = execv(cmd, args);

    printf("Execv returned %d\n", ret);

    return 0;
}
```

execv.c

```
$ ./execv
Hi!
ECS150
```

- Successful call to `exec()` never returns!
- Otherwise returns `-1` and continues current program if it fails

# Process management

## exit()

- Termination of the current process
- Ability to return an *exit value*

## Example

- Exit at any time during execution

```
if (error)
    exit(1);
```

- Or return from `main()`

```
int main(void) {
    ...
    return 0;
}
```

- Libc transparently exits

```
exit(main(argc, argv));
```

## Exit value usage

```
$ ls /
...
$ echo $?
0
```

```
$ ls /nodir
ls: cannot access '/nodir': No such file
or directory
$ echo $?
2
```

```
$ if [[ ! $(ls /nodir >& /dev/null) ]]; then echo "Expected..."; fi
Expected...
```



# Process management

## wait()/waitpid()

- `wait()` makes parent wait for *any* of its children to exit
  - Parent is blocked from execution in the meantime
- `waitpid()` enables more advanced options, such as
  - Specify PID of child to wait for
  - Don't block even if no children have exited yet

## Example

```
pid = fork();
if (pid != 0) {
    /* Parent */
    int status;
    wait(&status);
    /* == waitpid(pid, &status, 0) */
    printf("Child returned %d\n",
        WEXITSTATUS(status));
} else {
    /* Child */
    printf("Will exit soon!\n");
    exit(42);
}
```

wait.c

```
$ ./wait
Will exit soon!
Child returned 42
```

- Here, output order is guaranteed
- Constrained by blocking call

# Process management

## Putting it together: fork() + exec() + wait()

```
int main(void)
{
    pid_t pid;
    char *cmd = "/bin/echo";
    char *args[] = { cmd, "ECS150", NULL};

    pid = fork();
    if (pid == 0) {
        /* Child */
        execl(cmd, args);
        perror("execl");
        exit(1);
    } else if (pid > 0) {
        /* Parent */
        int status;
        waitpid(pid, &status, 0);
        printf("Child returned %d\n",
            WEXITSTATUS(status));
    } else {
        perror("fork");
        exit(1);
    }

    return 0;
}
```

fork\_exec\_wait.c

```
$ ./fork_exec_wait
ECS150
Child returned 0
```

## system()

- Somewhat equivalent to what function `system()` does internally

```
system("/bin/echo ECS150");
```

- But with a lot more control!

# Process management

## The shell

- A **shell** is a user interface to run commands in the terminal
- Typically makes heavy use of process-related functions

## Naive pseudo-implementation

```
while (1) {                                /* Repeat forever */
    char *command[ARG_MAX];

    printf("user@host $ ");                /* Display prompt in terminal */
    read_command(command);                  /* Read input from terminal */

    if (!fork()) {                          /* Fork off child process */
        exec(command);                      /* Execute command */
        perror("execv");                    /* Coming back here is an error */
        exit(1);
    } else {
        /* Parent */
        waitpid(-1, &status, 0);            /* Wait for child to exit */
    }
}
```

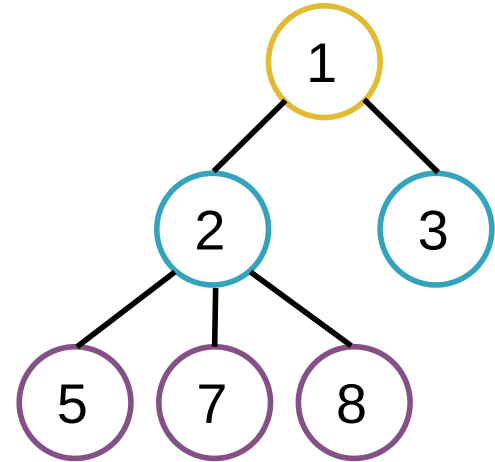
## Extra features

Background jobs (&), redirections (< and >) for connecting `stdin` or `stdout` of the child to files (instead of the terminal), pipes (|) for connecting `stdin` or `stdout` of the child to other processes, and many more.

# Process management

## getpid()/getppid()

- Notion of a (family) tree of processes
  - Only one parent per process
  - But possibly multiple children
- In Unix, `init` (PID=1) is ultimate ancestor
  - Only process created from scratch by kernel (and not by forking)
- `getpid()` returns process' PID
- `getppid()` return its parent's PID



## Example

```
int main(void)
{
    if (fork() > 0)
        /* Forces parent to wait for child
         * to force scheduling order */
        wait(NULL);

    printf("My PID is %d\n", getpid());
    printf("My parent's PID is %d\n", getppid());

    return 0;
}
```

getpid.c

```
$ ./getpid
My PID is 406782
My parent's PID is 406781
My PID is 406781
My parent's PID is 162474
```

```
$ echo $$
162474
```

# ECS 150 - System Calls

---

*Prof. Joël Porquet-Lupine*

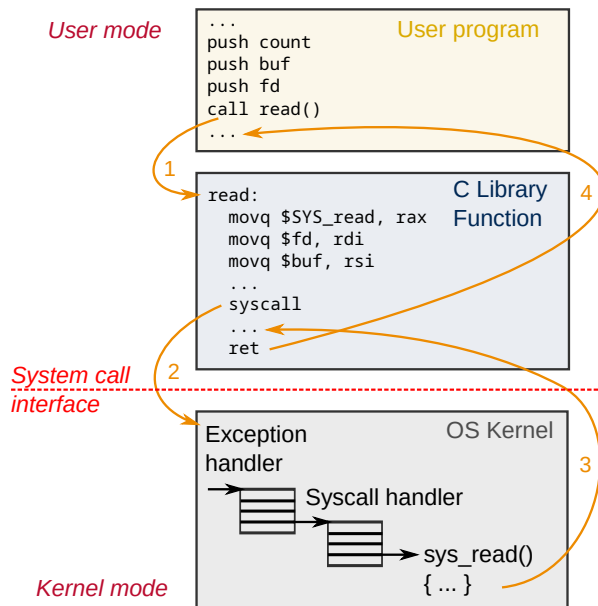
UC Davis - SQ25



# Recap

## C standard library

- Non-privileged functions
  - E.g., `memset()`
- Always/sometimes privileged functions
  - E.g., `read()/printf()`
  - Require **system call**



## Syscall categories

- **Process management**
  - A process is a program in execution
  - PCB data structure
- Files and directories, Pipes, Signals, Memory management

## Process management

- `fork()`
  - Clone process
- `exec()`
  - Execute different program inside current process
- `exit()`
  - Terminate current process
  - Return exit value
- `wait()/waitpid()`
  - Wait for child process to terminate
  - Collect termination status
- `getpid()/getppid()`
  - Identify process's PID or its parent's

# System calls

---

Process management

Files and directories

Pipes

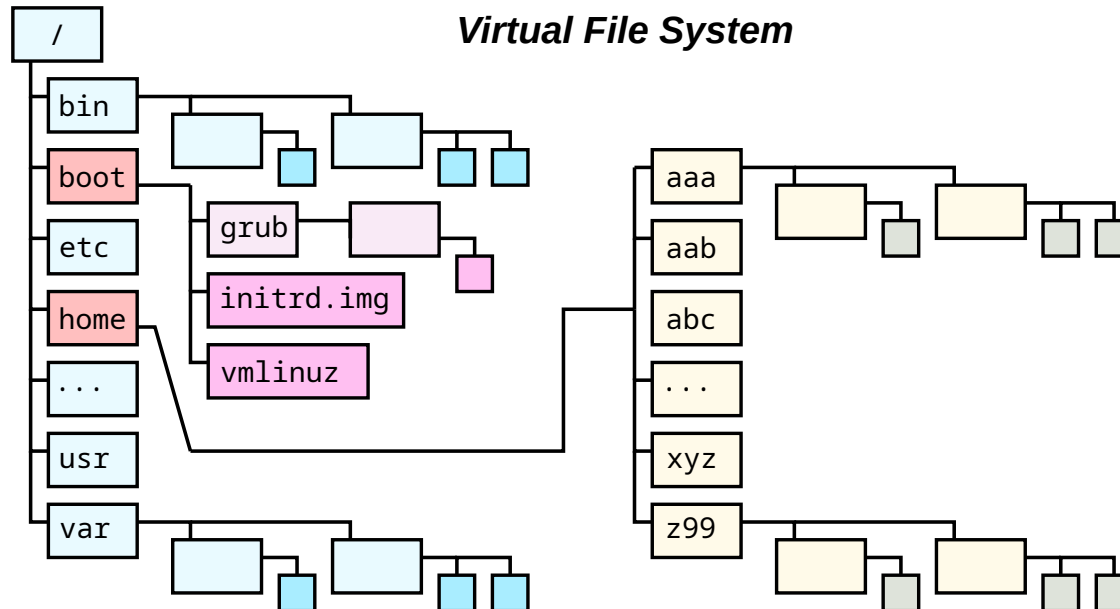
Signals

Memory management

# Files and directories

## Concepts

- Files and directories in tree-like structure called **Virtual File System**
  - Internal nodes are directories, leaf nodes are files
- Every directory contains a list of filenames
- Every file contains an array of bytes



- VFS can aggregate files and directories from various physical media (local hard-drive, remote network share, etc.)



# Files and directories

---

## Main related functions/syscalls

- File interaction
  - `open()`: open (create) file and return file descriptor
  - `close()`: close file descriptor
  - `read()`: read from file
  - `write()`: write to file
  - `lseek()`: move file offset
- File descriptor management
  - `dup2()`: duplicate file descriptor
- File characteristics
  - `stat()/fstat()`: get file status
- Directory traversal
  - `getcwd()`: get current working directory
  - `chdir()`: change directory
  - `opendir()`: open directory
  - `closedir()`: close directory
  - `readdir()`: read directory

# Files and directories

## Basic file interaction

- `open()` returns a **file descriptor** (*FD*), used for all interactions with file
  - Closed by `close()` when done with file
- `read()/write()` operations are sequential, tracked by file offset
- `lseek()` can manipulate current file offset

## Example

```
#include <fcntl.h>
...
int main(void)
...
    int fd;
...
    fd = open("file_101.c", O_RDONLY);

    read(fd, &c, 1);
    printf("%c\n", c);
    read(fd, &c, 1);
    printf("%c\n", c);

    lseek(fd, -2, SEEK_END);
    read(fd, &c, 1);
    printf("%c\n", c);

    close(fd);
...
}
```

file\_101.c

```
$ ./file_101
#
i
}
```

- Result
- Ask why `}` is one before last character
- Now, focus on file descriptor for transition to next slide
- `fd` is an `int` and is used as argument to all file manipulation after file was opened

# Files and directories

## File descriptors

### Definition

- Table of *open files* per process
  - Part of PCB
  - (Duplicated upon forking)
- FDs are simple indexes in the table

### Example

```
int fd1, fd2, fd3;

fd1 = open("file_101.c", O_RDONLY);
fd2 = open("file_201.c", O_RDWR);

printf("fd1 = %d\n", fd1);
printf("fd2 = %d\n", fd2);
```

file\_201.c

```
int fd1, fd2;
fd1 = open("file_101.c", O_RDONLY);
fd2 = open("file_201.c", O_RDWR);
```

Process

User

PCB

PID=X  
state  
ctxt  
files  
...

FD table

Mode / Offset / File

|     |    |   |   |
|-----|----|---|---|
| 0   |    |   |   |
| ... |    |   |   |
| 3   | RO | 0 | • |
| 4   | RW | 0 | • |
| ... |    |   |   |

file\_101.c

file\_201.c

Kernel

```
$ ./file_201
fd1 = 3
fd2 = 4
...
```

### Allocation

- `open()` always returns *first available* FD

```
close(fd1);
fd3 = open("file_201.c", O_WRONLY);
printf("fd3 = %d\n", fd3);
```

file\_201.c

```
$ ./file_201
...
fd3 = 3
```

# Files and directories

## Standard streams

Initially, three open file descriptors per process

- 0: standard input (STDIN\_FILENO)
- 1: standard output (STDOUT\_FILENO)
- 2: standard error (STDERR\_FILENO)

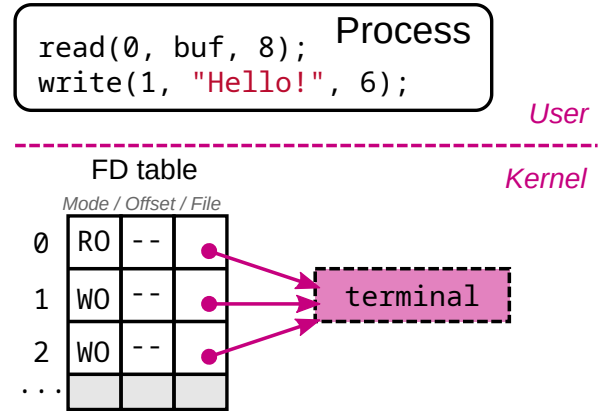
## Example

```
write(STDOUT_FILENO, "Hello ", 6);
write(1, "Hello ", 6);
fprintf(stdout, "Hello ");
printf("Hello\n");

write(STDERR_FILENO, "World ", 6);
write(2, "World ", 6);
fprintf(stderr, "World\n");
```

standard\_fds.c

```
$ ./standard_fds
Hello Hello Hello Hello
World World World
```



## Redirections

- Can connect standard streams to other targets than the terminal

```
$ ./standard_fds > /dev/null
World World World
$ ./standard_fds 2> /dev/null | tr H J
Jello Jello Jello Jello
```

```
$ ./standard_fds >& myfile.txt
$ cat myfile.txt
Hello Hello World World World
Hello Hello
```

# Files and directories

## File descriptor manipulation

- `dup2(src, dst)` replaces an open file descriptor (`dst`) with another (`src`)
  - *Note: avoid using deprecated `dup()`*

## Example

```
int main(void)
{
    int fd;

    printf("Hello #1\n");

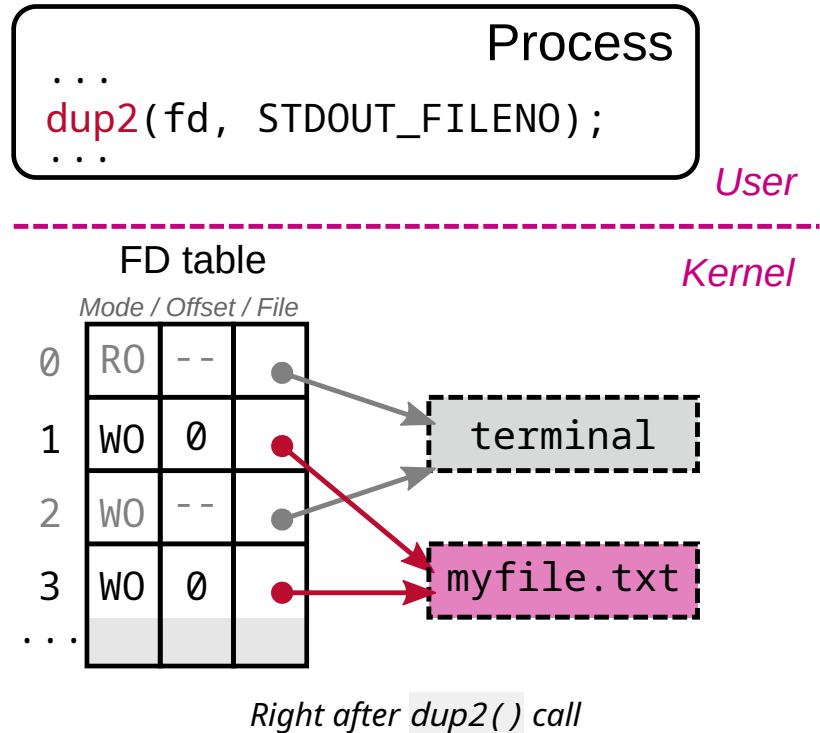
    fd = open("myfile.txt",
              O_WRONLY | O_CREAT,
              0644);
    dup2(fd, STDOUT_FILENO);
    close(fd);

    printf("Hello #2\n");

    return 0;
}
```

dup2.c

```
$ ./dup2
Hello #1
$ cat myfile.txt
Hello #2
```



# ECS 150 - System Calls

---

*Prof. Joël Porquet-Lupine*

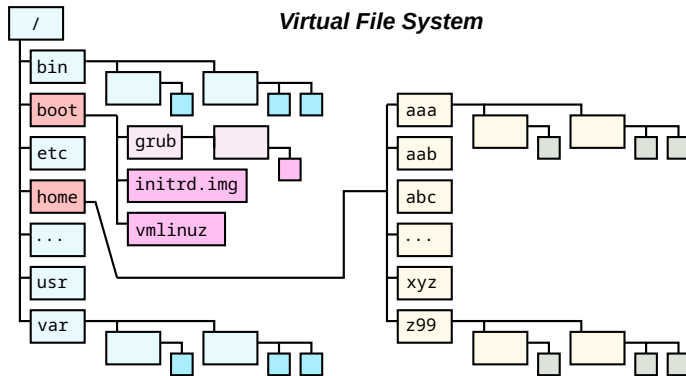
UC Davis - SQ25



# Recap

## Files and directories

### Virtual file system



### Basic file interaction

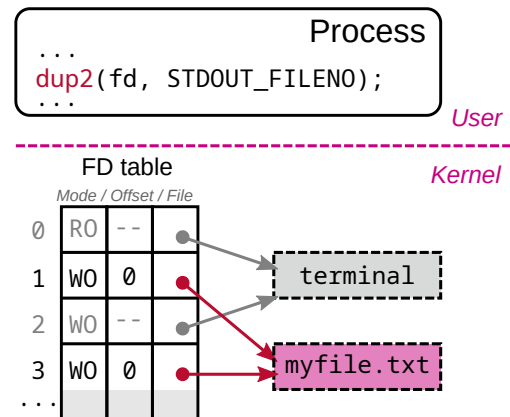
- `open()/close()`
  - Get/close file descriptor
  - File descriptor is an index in the process's file descriptor table
- `read()/write()`
  - Access file content, tracked by file offset
- `lseek()`
  - Move file offset to any location

### Standard streams

- `0 == STDIN_FILENO`
- `1 == STDOUT_FILENO`
- `2 == STDERR_FILENO`
- Connected to terminal by default, unless redirected

### `dup2()`

- Duplicate file descriptor
- E.g., can redirect standard stream from within running process



# Files and directories

## File characteristics

- `stat()/lstat()` return some information about a file, from a filename
- Same with `fstat()`, but from an FD

## Example

```
struct stat {
    dev_t    st_dev;        /* Defined in <sys/stat.h> */
    ino_t    st_ino;        /* ID of device containing file */
    mode_t    st_mode;      /* Inode number */
    nlink_t   st_nlink;     /* File type and mode */
    uid_t     st_uid;       /* Number of hard links */
    gid_t     st_gid;       /* User ID of owner */
    dev_t     st_rdev;      /* Group ID of owner */
    off_t     st_size;      /* Device ID (if special file) */
    blksize_t st_blksize;   /* Total size, in bytes */
    blkcnt_t  st_blocks;    /* Preferred blocks for I/O */
    struct timespec st_atim; /* Number of 512B blocks allocated */
    struct timespec st_mtim; /* Time of last access */
    struct timespec st_ctim; /* Time of last modification */
    /* Time of last status change */
};
```

```
int main(int argc, char *argv[])
{
    struct stat sb;

    stat(argv[1], &sb);

    printf("File type:          ");
    switch (sb.st_mode & S_IFMT) {
        case S_IFDIR:  printf("directory\n");    break;
        case S_IFREG:  printf("regular file\n");   break;
        default:       printf("other\n");          break;
    }
    printf("Mode:                %lo (octal)\n",
           (unsigned long) sb.st_mode);
    printf("File size:           %lld bytes\n",
           (long long) sb.st_size);
    printf("Last file access:    %s",
           ctime(&sb.st_atime));

    return 0;
}
```

stat.c

```
$ ./stat stat.c
File type:          regular file
Mode:               100644 (octal)
File size:          644 bytes
Last file access:   Fri Sep 18 16:32:02 2020
$ ./stat .
File type:          directory
Mode:               40755 (octal)
File size:          4096 bytes
Last file access:   Fri Sep 18 11:59:49 2020
```



# Files and directories

## Directory traversal

- `getcwd()/chdir()` to access the *current working directory*
- `opendir()/closedir()/readdir()` to access the entries of a directory

## Example

```
int main(int argc, char *argv[])
{
    char cwd[PATH_MAX];
    DIR *dirp;
    struct dirent *dp;

    getcwd(cwd, sizeof(cwd));
    printf("Change CWD from '%s' to '%s'\n",
           cwd, argv[1]);
    chdir(argv[1]);

    dirp = opendir(".");
    while ((dp = readdir(dirp)) != NULL)
        printf("Entry: %s\n", dp->d_name);
    closedir(dirp);

    return 0;
}
```

dir\_scan.c

```
$ pwd
/home/jporquet
$ ./dir_scan /
Change CWD from '/home/jporquet' to '/'
Entry: ..
Entry: lib
Entry: home
Entry: etc
Entry: root
[...]
Entry: proc
Entry: tmp
Entry: dev
Entry: bin
Entry: sbin
Entry: mnt
Entry: .
Entry: usr
```

# System calls

---

Process management

Files and directories

Pipes

Signals

Memory management

# Pipe

## Definition

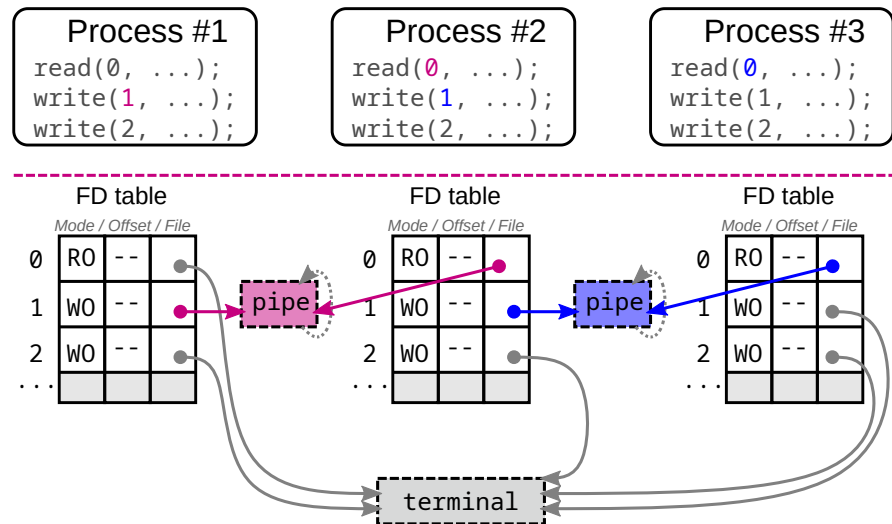
- Inter-process communication (IPC)
- Pipeline of processes chained via their standard streams
  - `stdout` of one process connected to `stdin` of next process

## Example

```
$ cd ~/teach
$ du -sh * | sort -h -r | head -3
1.2G    ecs150
555M    ecs36c
386M    ecs30
```

## Details

- Internally implemented as anonymous files
  - Circular memory buffer of fixed size
  - Accessible via file descriptors and regular read/write transfers
- Processes run *concurrently*, implicitly synchronized by communication



# Pipe

## pipe()

- Create a pipe and return two file descriptors via array
  - [0] for reading access, [1] for writing access

## Example

```
int main(void)
{
    int fd[2];
    char send[7] = "Hello!";
    char recv[7];

    pipe(fd);
    printf("fd[0] = %d\n", fd[0]);
    printf("fd[1] = %d\n", fd[1]);

    write(fd[1], send, 7);

    read(fd[0], recv, 7);
    puts(recv);

    return 0;
}
```

pipe.c

```
$ ./pipe
fd[0] = 3
fd[1] = 4
Hello!
```

## Process

```
int fd[2];
pipe(fd);
```

## FD table

|     | Mode | Offset | File |
|-----|------|--------|------|
| 0   | RO   | --     | •    |
| 1   | WO   | --     | •    |
| 2   | WO   | --     | •    |
| 3   | RO   | --     | •    |
| 4   | WO   | --     | •    |
| ... |      |        |      |

terminal

pipe

# Pipe

## Process pipeline example

- Pseudo-code setting up two children as `command1 | command2`

```
void pipeline(char *command1, char *command2)
{
    pid_t p1, p2;
    int fd[2];

    pipe(fd);

    if (!(p1 = fork())) {          /* Child #1 */
        close(fd[0]);              /* No need for read access */
        dup2(fd[1], STDOUT_FILENO); /* Replace stdout with pipe */
        close(fd[1]);              /* Close now unused FD */
        exec(command1);            /* Child #1 becomes command1 */
    }

    if (!(p2 = fork())) {          /* Child #2 */
        close(fd[1]);              /* No need for write access */
        dup2(fd[0], STDIN_FILENO); /* Replace stdin with pipe */
        close(fd[0]);              /* Close now unused FD */
        exec(command2);            /* Child #2 becomes command2 */
    }

    close(fd[0]);                  /* Pipe no longer needed */
    close(fd[1]);
    waitpid(p1, NULL, 0);          /* Waits for children */
    waitpid(p2, NULL, 0);
}
```

# ECS 150 - System Calls

---

*Prof. Joël Porquet-Lupine*

UC Davis - SQ25



# Recap

## Files and directories

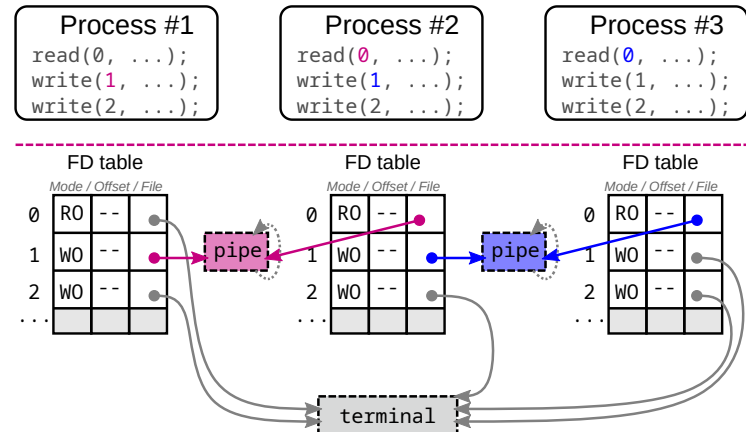
### File characteristics

- `stat()/fstat()`: information about file
- `getcwd()/chdir()`: access the *current working directory*
- `opendir()/closedir()/readdir()`: access the entries of a directory

## Pipes

- Typically used to chain processes to one another and create complex commands
  - Example: get three biggest items within current directory

```
$ du -sh * | sort -h -r | head -3  
...
```



# System calls

---

Process management

Files and directories

Pipes

Signals

Memory management



# Signals

## Definition

- Form of inter-process communication (IPC)
- Software notification system
  - From process' own actions: e.g., SIGSEGV (*Segmentation fault*)
  - From external events: e.g., SIGINT (*Ctrl-C*)
- About 30 different signals (see `man 7 signal`)

## Default action

In case process does not define specific signal handling

- Terminate process: e.g., SIGINT, SIGKILL<sup>\*</sup>
- Terminate process and generate core dump: e.g., SIGSEGV
- Ignore signal: e.g., SIGCHLD
- Stop process: e.g., SIGSTOP<sup>\*</sup>
- Continue process: e.g., SIGCONT

## Handling or ignoring

- Possible to change default action (<sup>\*</sup> not for all signals!)
1. Ignore signals
    - Mask of blocked signals
  2. Set signal handlers
    - Function to be run upon signal delivery

# Signals

---

## Main related functions/syscalls

- Sending signals
  - `raise()`: Send signal to self
  - `kill()`: Send signal to other process
  - `alarm()` / `setitimer()`: Set timer for self
    - Receive signal (`SIGALRM`/`SIGVTALRM`) when timer is up
- Blocking signals
  - `sigprocmask()`: Examine or change signal mask
  - `sigpending()`: Examine pending blocked signals
- Receiving signals
  - `sigaction()`: Map signal handler to signal
    - Also `signal()` but not recommended
  - `pause()`: Suspend self until signal is received

# Signals

## Example

```
void alarm_handler(int signum)
{
    printf("\nBeep, beep, beep!\n");
}

int main(void)
{
    struct sigaction sa;
    sigset_t ss;

    /* Ignore Ctrl-C */
    sigemptyset(&ss);
    sigaddset(&ss, SIGINT);
    sigprocmask(SIG_BLOCK, &ss, NULL);

    /* Set up handler for alarm */
    sa.sa_handler = alarm_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGALRM, &sa, NULL);

    /* Configure alarm */
    printf("Alarm in 5 seconds...\n");
    alarm(5);

    /* Wait until signal is received */
    pause();

    /* Bye, ungrateful world... */
    raise(SIGKILL);

    return 0;
}
```

signal.c

```
$ ./signal
Alarm in 5 seconds...
^C^C^C^C
Beep, beep, beep!
zsh: killed ./signal
```

# System calls

---

Process management

Files and directories

Pipes

Signals

Memory management

# Memory

---

## Division of labor

### User C library

- `malloc()`/`free()` for dynamic memory allocation
  - **Heap** memory segment (at the end of **data** segment)
- Fine-granularity management only
  - When heap is full, syscall to kernel to request for more

### OS/Kernel

- Memory management at "page" level
- Allocation of big chunks (many pages) to user library

## Related functions/syscalls

### `sbrk()`/`brk()`

- Linear increase of data segment
- Old way of allocating heap space <sup>1</sup>
- Legacy function now

### `mmap()`

- Map pages of memory in process' address space
- Can also map a file's contents
- Extremely versatile and powerful function

<sup>1</sup> <https://utcc.utoronto.ca/~cks/space/blog/unix/SbrkVersusMmap>